

1 Week 19: Docker、CI/CD 与部署

1.1 本周目标

本周处于“生产化”阶段。完成后，你应能说明并实践：Docker Compose、健康检查、CI；同时交付 容器化、可部署的 Agent Web 产品。

1.2 对应岗位能力

将用户问题拆成可验证的技术任务；实现小而清晰的模块；对失败路径负责；以文档和验收标准说明工程决策。

1.3 每日任务总览

日程	主题	建议投入
Day 1	Docker、CI/CD 与部署：概念与最小示例	2-3 小时
Day 2	Docker、CI/CD 与部署：基础编码练习	2-3 小时
Day 3	Docker、CI/CD 与部署：功能开发	2-3 小时
Day 4	Docker、CI/CD 与部署：工程化改造	2-3 小时
Day 5	Docker、CI/CD 与部署：项目开发	2-3 小时
Day 6	Docker、CI/CD 与部署：独立挑战与问题修复	2 小时
Day 7	Docker、CI/CD 与部署：复习、整理和自由补充	2 小时

1.4 时间分配

建议投入 **15-20 小时**：概念与阅读 20%，编码与测试 45%，项目集成 20%，复盘与补做 15%。

1.5 必须掌握

Docker Compose、健康检查、CI。

1.6 需要理解

环境分层。

1.7 暂时了解

Kubernetes。此部分不应阻塞本周项目。

1.8 本周编码任务

在 `projects/project-04-capstone/` 中完成一个可运行的最小功能，并至少新增一个正向测试和一个异常/边界测试。代码需要类型提示、异常处理和可读日志；密钥只能从环境变量读取。

1.9 本周项目里程碑

项目：案例分流与合规助手。本周里程碑：容器化、可部署的 Agent Web 产品。将实际实现状态记录到项目 `TASKS.md`。

1.10 工程要求

- 所有新增 Python 函数必须有类型提示。
- 关键输入必须验证；异常不能被静默吞掉。
- README 中的命令必须经过一次实际运行验证。
- 提交前运行测试，并在提交信息中使用英文动词短语。

1.11 验收标准

- [] 本周最小功能可以运行。
- [] 至少包含两类测试样例。
- [] 关键错误被记录并可理解地返回。
- [] 没有提交真实 API Key、令牌或个人数据。
- [] README、学习日志和项目任务表已更新。

1.12 常见风险

风险	预防或处理
追逐新框架而没有完成最小闭环	先完成本周验收，再把新框架列入“暂时了解”。
只展示正常路径	在实现前先写一个无效输入或外部调用失败的测试。
设计叙事替代工程证据	用运行命令、测试结果和日志字段作为证据。

1.13 补做任务

若时间不足，优先保留：最小功能、一个测试、README 运行说明和学习日志。其余优化进入下一周的 Day 7。

1.14 提前完成后的进阶任务

为本周功能增加一个可观测性字段、一个输入边界测试，或写一段“教学版与生产版”的差异说明。

1.15 本周面试问题

请用 2 分钟回答：为什么选择当前的数据结构、API 契约或工作流边界？它在输入错误、外部服务失败和需要人工确认时如何表现？

1.16 周末复盘方式

阅读 REVIEW.md, 从“完成事实、失败样例、工程改进、下周风险”四个角度各写一段。不要只记录感受。

1.17 下周准备事项

阅读 Week 20 的 WEEK_PLAN.md, 检查本地运行环境、未完成任务与需准备的非敏感示例数据。

1.18 参考资源索引